



Autómatas, Teoría de Lenguajes y Compiladores

PROYECTO ESPECIAL

G-113 (PICTURE)

Integrantes:

- Augusto Andrés Lanari (64679)
- Jerónimo Lopez Vila (65778)
- Luca Mancuso (64150)
- Santiago Torres (64563)

Profesores:

- Ana Arias Roig
- Mario Agustin Golmar

Fecha de entrega: 28/11/2025

PROYECTO ESPECIAL	0
1. Introducción	2
2. Modelo Computacional	2
2.1. Dominio	2
2.2 Construcciones	3
3. Implementación	4
3.1 Frontend	4
3.2. Backend	4
3.3 Dificultades Encontradas	5
4. Usando PICTURE; virtudes y limitaciones	5
5. Futuras Extensiones	7
6. Conclusiones	7
7. Bibliografía	8

1.Introducción

El siguiente informe detalla el desarrollo realizado por el grupo mencionado. El objetivo de PICTURE es permitir que los desarrolladores definan interfaces gráficas para el sistema operativo PinkOS, mediante descripciones textuales, de forma más sencilla que escribir la lógica en C. Un compilador/transpilador convierte estas descripciones a código C que utiliza las primitivas y un runtime de PinkOS para renderizar los elementos y manejar eventos. El formato de PICTURE se asemeja a html en su concepción, a python o yaml en su estética y a otros lenguajes ampliamente usados por la industria, con el objetivo de una fácil adaptación de los programadores.

El compilador fue desarrollado en el lenguaje C, usando las herramientas Flex y Bison proveídas por la cátedra. Asimismo, constó el desarrollo en tres etapas o *stages*. En la primera se realizaron las especificaciones del dominio y construcciones de lo propuesto; en la segunda, se realizó el *frontend*, la tokenización de lo escrito para el análisis sintáctico de lo escrito en este lenguaje y su evaluación sobre su validez; La tercera y última, fue el desarrollo del *backend*, que consistió en el análisis semántico de lo escrito en el lenguaje y su transcripción a lenguaje C, para ser usado por PinkOS.

2.Modelo Computacional

El Modelo computacional fue definido en la *stage* I. se transcribe:

2.1. Dominio

PinkOS es un sistema operativo experimental desarrollado en el marco de las materias Arquitectura de Computadoras y Sistemas Operativos. Para facilitar el desarrollo de interfaces gráficas en PinkOS, se propone la creación de un lenguaje declarativo llamado PICTURE (PinkOS Interface Construction Terminology for UI Representation & Events).

El objetivo de PICTURE es permitir que los desarrolladores definan interfaces gráficas mediante descripciones textuales, de forma más sencilla que escribir la lógica en C. Un compilador/transpilador convierte estas descripciones a código C que utiliza las primitivas de PinkOS para renderizar los elementos y manejar eventos.

2.2 Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

(algunas fueron modificadas por limitaciones técnicas del runtime de PinkOS)

- (I). Toda la interfaz se define como un árbol jerárquico de componentes, donde uno se define dentro del otro usando la tabulación.
- (II). Cada componente requiere un identificador único dentro de su ventana, definido con #, el cual permitirá referenciarlo desde la lógica en C.
- (III). Cada componente tiene propiedades que definen características de estilo, posicionamiento, alineamiento, etc...
- (IV). El valor de las propiedades de los componentes puede ser de tipos de dato variados, incluyendo algunos built-ins como "s", "m" y "l" para tamaño de texto o "blue", "red", etc...para colores.
- (V). Se pueden definir variables globales para usar como valor en las propiedades de los componentes, que en principio son más como defines en C ya que son un reemplazo textual.
- (VI). Las propiedades pueden modificarse desde la lógica en C, haciendo posible UIs dinámicas.
- (VII). El orden de render (z-index) se deriva de la posición en el árbol.
- (VIII). Los componentes pueden definir callbacks que determinan qué lógica se asociará acciones dadas (on_press, on_select y on_keypress).
- (IX). Los componentes seleccionables (o sea que hayan definido algún callback que dependa de tener el foco) se iterarán con tab en el orden especificado en la metadata (headers de PICTURE definidos al comienzo).
- (X). Hacer enter o escribir sobre un componente en foco ejecutará el callback correspondiente.
- (XI). Se permitirá escribir comentarios con //.

3. Implementación

3.1 Frontend

Esta etapa incluyó el análisis léxico y sintáctico, para ello se tokenizó lo escrito así como la gestión de buffers de entrada y la creación de estructuras internas para representar los tokens y las reglas gramaticales. Todo esto a partir de lo descrito y pensado en stage I. Asimismo, se hicieron los test correspondientes, tanto de aceptación como de rechazo de lo escrito.

Análisis Léxico

El análisis léxico se implementó utilizando la herramienta Flex, que permite definir patrones para identificar los tokens del lenguaje. Cada token representa una unidad léxica del lenguaje, palabras clave, operadores, etc. Se definieron también funciones necesarias para gestionar este analizador incluyendo la creación y destrucción de buffers de entrada y la inicialización para el análisis.

Análisis Sintáctico

El análisis sintáctico se implementó utilizando Bison, para definir la gramática del lenguaje y generar un analizador sintáctico que valida la estructura de lo escrito. En esta etapa, se verifican las reglas gramaticales del lenguaje PICTURE y se generan estructuras internas que representan, el árbol de sintaxis, que representa la jerarquía de componentes y sus propiedades.

3.2. Backend

El backend se encarga de realizar el análisis semántico y la generación de código C. Esta etapa toma como entrada las estructuras generadas por el frontend y las procesa para validar las reglas semánticas del lenguaje y generar el código correspondiente.

Análisis Semántico

El análisis semántico verifica que las descripciones textuales cumplan con las reglas semánticas del lenguaje PICTURE. Esto incluye la validación de identificadores únicos, tipos de datos, y la coherencia de las propiedades de los componentes. Si se detecta algún error, el compilador informa el problema y aborta la generación de código.

Generación de Código

Una vez se valida, inicia la generación de código, que traduce el código de PICTURE a código C que usará luego PinkOS para renderizar los elementos y manejar eventos. El código generado es compatible con el sistema operativo PinkOS.

3.3 Dificultades Encontradas

Se debió modificar algunas cuestiones realizadas en la etapa de *frontend* para que la generación de código sea compatible con cambios realizados en PinkOS, dado que se prioriza la compatibilidad para que ya el lenguaje sea utilizable.

El desarrollo del compilador en sí no presentó problemas significativos más allá de la complejidad natural del diseño del AST, la gestión de memoria y la integración entre Flex y Bison.

Si bien está por fuera del eje del desarrollo de este trabajo, la principal dificultad que se encontró fue el diseño e implementación del sistema gráfico para PinkOS, debido a que se quería algo funcional para poder testear el output de PICTURE y terminar esta etapa de desarrollo con un MVP que sea funcional para al menos desarrollar UIs sencillas, con la intención de ya poder usar PICTURE para las necesidades gráficas de nuestro sistema operativo, pensando en que vaya creciendo a medida que los requisitos de nuestras UIs lo demanden.

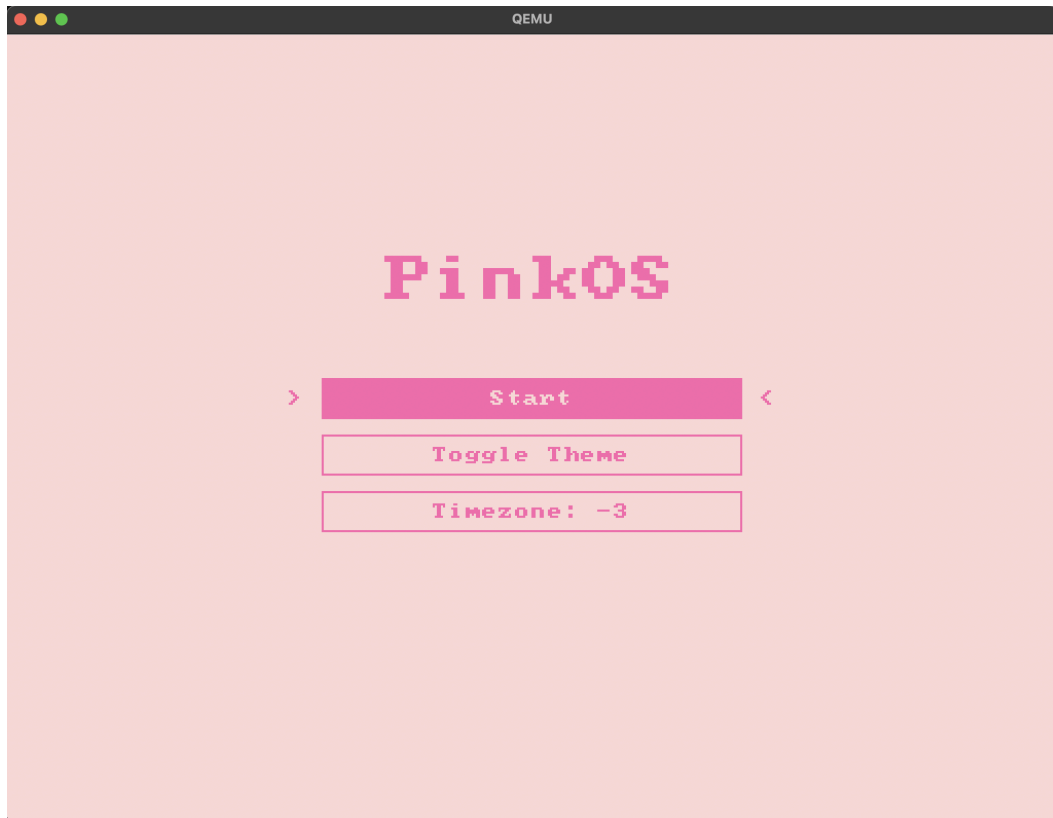
Actualmente estamos contentos de que con las funcionalidades a las que llegamos logramos programar la home screen gráfica que teníamos diseñada desde Arqui pero por limitaciones técnicas no habíamos implementado (ver sección 4 o ejemplos en github).

4. Usando PICTURE; virtudes y limitaciones

El lenguaje PICTURE fue concebido para facilitarnos la creación de interfaces en PinkOS, por ende lo natural a la hora de probarlo fue escribir algunas UIs testeando cada funcionalidad y buscando sus limitaciones.

Pudimos crear UIs que tengan botones que modifiquen elementos, campos de texto incluso con restricciones (ej. solo numérico), teclas que modifican propiedades como tamaño o alineamiento de elementos, y hasta parámetros animados.

Esto nos permitió programar con nuestro framework la home screen de PinkOS que incluye varias de esas interacciones y permite configurar cosas del sistema como el tema o zona horaria, y tiene el estilo visual del resto del OS:



Ver demo en video en el repo de PICTURE

De todas formas vimos limitaciones a la hora de hacer propiedades que sean referencia a otra propiedad donde modificar una modifique la otra, o hacer propiedades que apliquen alguna clase de operación aritmética con los inputs, ya que eso implicaría en C tipos de dato complejos y un desarrollo que no pudo completarse.

Igualmente muchas de estas limitaciones pueden sortearse con trabajo del lado del que programa la lógica (ya que en runtime puede modificar los valores, y puede implementar la lógica que quiera aunque no tenga ayudas desde el lenguaje PICTURE), y tenemos el diseño de soluciones que permitirían a futuro implementar estas funciones (ver `variables-propiedades-y-PictureValues.md` en la carpeta doc del repo).

Otra limitación es que no se dispone de un componente de tipo canvas o algo similar que permita al programador dibujar cosas a medida que no sean soportadas por PICTURE, lo cual limita bastante el tipo de UIs que se pueden hacer; está enfocado más bien a interfaces de texto/configuración, lo cual es útil pero tiene un scope reducido.

5. Futuras Extensiones

Algunas extensiones posibles incluyen:

- Permitir propiedades más complejas (layouts avanzados, estilos anidados, constraints).
- Desarrollar el sistema de PictureValues que permitiría variables component-scoped, propiedades relativas al componente padre, propiedades que hagan referencia a propiedades de otros componentes o incluso expresiones calculadas en runtime (ver `variables-propiedades-y-PictureValues.md` en la carpeta doc del repo).
- Agregar callbacks adicionales y eventos más ricos.
- Permitir composición y reutilización de componentes.
- Permitir que un componente tenga un Bitmap para el fondo, que actúe como “canvas” donde el programa pueda dibujar cosas a medida como gráficos, imágenes, etc. (etc con un solo punto, je).
- Desarrollar un Language Server (LSP)
- Crear una extensión para VS Code que ofrezca resaltado de sintaxis, colores, soporte para comentarios y snippets.

6. Conclusiones

El desarrollo del lenguaje y compilador PICTURE cumplió satisfactoriamente los objetivos planteados. El lenguaje permite describir interfaces de manera clara y declarativa, y aunque en principio es sencillo ya nos fue útil para un caso de uso real en PinkOS.

El proyecto permitió afianzar conceptos sobre el análisis léxico, sintáctico y semántico, así como la interacción de estos componentes dentro de un compilador completo, y nos permitió realizar una idea que pensábamos fantástica que es programar nuestro propio OS con nuestro propio lenguaje.

7. Bibliografía

<https://wiki.osdev.org/GUI>

https://en.wikipedia.org/wiki/Retained_mode

- ☰ (2025-08-21, v3.0.8) Proyecto Especial
- ☰ (2025-03-13, v0.1.0) Modelo Computacional
- ☰ (2025-09-30, v1.0.1) Especificación
- ☰ (2025-09-03, v1.0.0) Análisis Léxico
- ☰ (2024-05-08, v0.1.0) Análisis Sintáctico
- ☰ (2024-09-25, v0.2.0) Análisis Semántico
- ☰ (2024-05-16, v0.1.0) Generación de Código
- ☰ (2024-05-16, v0.1.0) Runtime